Module 4: Evaluation

Runtime Semantics

Let the execution begin!

1.

Syntax vs Semantics

AKA Appearance vs Personality

Syntax vs Semantics

Syntax

Syntax describes the textual structure of a language (in effect, the grammar).

- A variable declaration looks like:
  - `LET identifier = expr;`
- Good syntax helps quickly identify meaning based on the text alone.
  - Is `name = expr;` a variable declaration or assignment?

Semantics

Semantics describe the actual meaning of code independent from how it is written.

- A variable declaration actually means:
  - Define “identifier” to have the value “expr”.
- Semantics cover all of the critical behavior:
  - What restrictions beyond syntax apply?
    - what if “identifier” is already defined?
  - What is the actual runtime behavior?
    - evaluate “expr”, assign to “identifier”.
- A language’s “power” comes from semantics.

What does the following evaluate to?

TRUE OR FALSE AND FALSE

Well, that depends on semantics!

What about SQL?

```
SELECT "Divide By Zero!" WHERE 1=1 OR 1/0=0
```

SQL doesn’t guarantee short circuiting!

2.

Runtime Semantics

How do we define what the code actually does?

Evaluation Order

Definitions

Evaluation Order defines the order that stmts/exprs are evaluated in at runtime.

- Evaluation *often* proceeds left to right, top to bottom, but not always!
  - “short-circuiting” operators, like &&/||, may not evaluate the RHS.
  - Loops, break/continue, return, & exceptions change control flow.
- In C++, function argument order is undefined, e.g. f(print(1), print(2)).

```java
public class Main {
  public static void main(String[] args) {
    for (int i = 1; i <= 100; i++) {
      if (i % 3 == 0 && i % 5 == 0) {
        System.out.println("FizzBuzz");
      } else if (i % 3 == 0) {
        System.out.println("Fizz");
      } else if (i % 5 == 0) {
        System.out.println("Buzz");
      } else {
        System.out.println(i);
      }
    }
  }
}
```

Scoping

Definitions

A **Scope** binds values to an identifier for recurring use throughout a program.

- Variables, functions, and classes all need to be “in-scope” to be used!
  - Imports add classes to the scope.
  - Defining what’s “in-scope” varys and has a range of consequences.
- Scopes can be *nested*:
  - Blocks can use outer variables.
  - Methods can access class fields.

```java
public class Main {
  public static void main(String[] args) {
    for (int i = 1; i <= 100; i++) {
      if (i % 3 == 0 && i % 5 == 0) {
        System.out.println("FizzBuzz");
      } else if (i % 3 == 0) {
        System.out.println("Fizz");
      } else if (i % 5 == 0) {
        System.out.println("Buzz");
      } else {
        System.out.println(i);
      }
    }
  }
}
```

What semantics would Functions have?

`function(first, second, third, …)`

1. Function is defined somewhere in scope.
2. Function has the right number of arguments.
3. Arguments are evaluated left to right.
4. Thrown exceptions change control flow.

Module 4: Evaluation

Runtime Semantics

Let the execution begin!

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals